

PYTHON SECURITY TOOLKIT

An Integrated Ethical Hacking Framework

PROJECT REPORT

on

“Python Security Toolkit: An Integrated Ethical Hacking Framework”

Technology: Python

Project Type: Cybersecurity / Security Testing Toolkit

Development Environment: Windows 11 + PyCharm

Testing Environment: Authorized and controlled laboratory environment

SUBMITTED BY

Srushti D. Porwal

Class: TE

Department: Computer Engineering

College: Marathwada Mitra Mandal's Institute Of Technology, Pune-411 047

Academic Year

2026–2027

Contents

1. Abstract
2. Objectives
3. Tools and Technologies
4. System Architecture
5. Module Documentation
6. Testing and Validation
7. Results and Screenshots
8. Challenges and Improvements
9. Limitations
10. Conclusion
11. Responsible Use
12. References

Abstract

The Python Security Toolkit is an integrated Python-based cybersecurity framework developed to organize multiple security-testing and defensive demonstration modules under a single command-line interface.

The toolkit integrates modules for MAC address changing, network scanning, ARP spoofing demonstration, packet sniffing demonstration, network queue demonstration, DNS spoofing demonstration, and ARP spoof detection. The project focuses on modular organization, reusable utilities, input validation, exception handling, testing, and clear user interaction.

A central menu allows the user to select a module and provide the required inputs. The toolkit validates these inputs and provides appropriate status, result, or error messages. Potentially disruptive functionality is implemented as safe demonstrations, while the ARP spoof detector provides a defensive anomaly-detection capability.

The project is designed for cybersecurity education and authorized laboratory testing. It emphasizes responsible use, controlled environments, documentation, and understanding of the underlying security concepts rather than unauthorized activity.

Objectives

The main objectives of the Python Security Toolkit are:

1. To integrate the cybersecurity scripts developed during the bootcamp into a single organized toolkit.
2. To provide a common command-line interface for selecting and using different security modules.
3. To organize the project into reusable and modular Python components.
4. To implement input validation and exception handling for safer and more reliable execution.
5. To remove unnecessary hard-coded environment values and allow users to provide required inputs.
6. To provide clear status, success, warning, and error messages during execution.
7. To test the individual components and document the testing process.
8. To demonstrate cybersecurity concepts in an authorized and controlled laboratory environment.
9. To provide proper documentation, screenshots, dependencies, and responsible-use guidelines.
10. To develop a coherent cybersecurity toolkit that demonstrates understanding of security testing, defensive detection, software organization, and responsible cybersecurity practices.

Tools and Technologies

Programming Language

- Python 3.13.0

Development Environment

- Windows 11
- PyCharm

Python Libraries and Tools

- Scapy — used for network-related security demonstrations and ARP-based scanning.
- Pytest — used for automated testing of project components.
- Npcap — installed on Windows to provide the packet-capture/network access required by Scapy.

Project Components

The toolkit is organized into separate Python modules for:

- MAC Address Changer
- Network Scanner
- ARP Spoofing Demo
- Packet Sniffer Demo
- Network Queue Demo
- DNS Spoofing Demo• ARP Spoof Detector

Additional utility modules are used for validation, network-related helper functions, and logging.

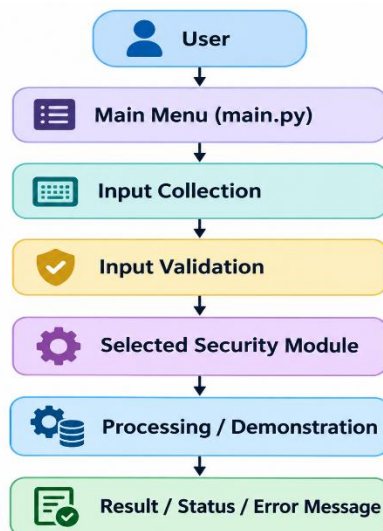
Project Structure

The project follows a modular structure with a central main.py file, separate security modules, reusable utility functions, automated tests, screenshots, and report documentation.

System Architecture

The Python Security Toolkit follows a modular architecture in which `main.py` acts as the central controller. The user interacts with the toolkit through a common command-line menu and selects the required security module. The selected module receives the required user inputs, validates them, performs its defined demonstration or defensive function, and returns the result to the main interface.

Architecture Flow



Module Layer

The toolkit contains seven security-related modules:

- `mac_changer.py`
- `network_scanner.py`
- `arp_spoof.py`
- `packet_sniffer.py`
- `network_jammer.py`
- `dns_spoof.py`
- `arp_spoof_detector.py`

Utility Layer

Reusable supporting functions are placed in the `utils` directory:

- `network_utils.py` — common network-related helper functions.
- `validators.py` — input validation functions.
- `logger.py` — timestamped toolkit messages.

Testing Layer

The `tests` directory contains automated tests used to verify individual components of the toolkit.

This modular architecture separates the user interface, security modules, reusable utilities, and testing components, making the project easier to understand, maintain, and extend.

Module Documentation

1. MAC Address Changer

The MAC Address Changer module accepts a network interface name and a requested MAC address. It validates the MAC address format before processing the request.

For this project, actual adapter modification is disabled. The module demonstrates input validation and clearly informs the user that the requested change is not performed.

2. Network Scanner

The Network Scanner uses Scapy to perform an ARP-based scan of an authorized IP address or network range.

The user provides the target range, which is validated before the scan begins. Discovered devices are displayed with their IP and MAC addresses.

3. ARP Spoofing Demo

The ARP Spoofing module demonstrates the inputs required for an ARP spoofing scenario, including target IP, gateway IP, and network interface.

The project version does not transmit ARP spoofing packets. It validates the supplied information and provides a safe demonstration of the concept.

4. Packet Sniffer Demo

The Packet Sniffer module accepts a network interface and packet limit as inputs.

The current project version demonstrates the required configuration and validation without performing live packet capture. This prevents collection of real network traffic during the demonstration.

5. Network Queue Demo

The Network Queue module accepts a network interface and packet limit.

It demonstrates the configuration and validation required for network queue processing. Live packet blocking, dropping, or modification is disabled in the project version.

6. DNS Spoofing Demo

The DNS Spoofing module accepts a target IP address, spoof IP address, and network interface.

The supplied IP addresses are validated before execution. Live DNS interception or modification is disabled, making the module suitable for controlled educational demonstration.

7. ARP Spoof Detector

The ARP Spoof Detector is the defensive component of the toolkit.

It analyses ARP observations and checks whether the same IP address is associated with conflicting MAC addresses. When a conflict is detected, the module reports it as a possible ARP spoofing indicator.

The detector was tested using controlled sample observations containing a conflicting MAC address.

Testing and Validation

The toolkit was tested at both the individual module level and the overall application level.

Functional Testing

The central menu was tested to verify that all seven modules could be selected successfully and that the Exit option returned the user from the application.

Each module was also tested with the required inputs to verify input collection, validation, and expected output.

Input Validation Testing

The project includes validation for important user inputs such as:

- MAC address format
- IP addresses
- Network ranges
- Network interface names
- Numeric packet limits

Invalid inputs were tested to confirm that appropriate error messages are displayed instead of allowing invalid values to continue.

Automated Testing

Pytest was used to test the MAC address validation utility.

The automated test cases included:

- A valid colon-separated MAC address
- An invalid MAC address
- A valid hyphen-separated MAC address Test result:

3 tests passed successfully.

Network Scanner Testing

The Network Scanner was tested using an authorized local network range. On Windows, Npcap was installed to provide the network access required by Scapy for the ARP-based scan.

ARP Spoof Detector Testing

The ARP Spoof Detector was tested using controlled sample observations. A conflicting MAC address was intentionally provided for an existing IP address, and the detector successfully reported the conflict as a possible ARP spoofing indicator.

Safety Testing

Potentially disruptive modules were implemented as demonstrations rather than performing live attack operations. Packet capture, ARP spoofing, packet blocking/jamming, and DNS interception/modification remain disabled in the current project implementation.

All testing was performed for educational purposes within an authorized environment.

Results and Screenshots

The integrated toolkit was successfully executed through the central command-line interface. All seven modules were connected to the main application and tested with their required inputs.

The screenshots below provide evidence of the toolkit menu and individual module execution.

Figure 1 — Main Toolkit Menu

The main menu provides access to all seven security modules and the Exit option.

```
C:\cyberboot\python_security_toolkit\.venv\Scripts\python.exe C:\cyberboot\python_security_toolkit\main.py
WARNING: No libpcap provider available ! pcap won't be used

=====
          PYTHON SECURITY TOOLKIT
    Integrated Ethical Hacking Framework
=====
[2026-09-16 01:10:23] [INFO] Python Security Toolkit started

=====
          MODULE MENU
=====
[1]  MAC Address Changer
[2]  Network Scanner
[3]  ARP Spoofing Demo
[4]  Packet Sniffer
[5]  Network Queue Demo
[6]  DNS Spoofing Demo
[7]  ARP Spoof Detector
[8]  Exit
=====
Select an option:
```

Figure 1 — Main Toolkit Menu

Figure 2 — MAC Address Changer

The module validates the supplied interface and MAC address and displays the status of the demonstration.

```
Select an option: 1

[INFO] MAC Address Changer selected.
Enter network interface name: Wi-Fi
Enter new MAC address: 00:11:22:33:44:55

[*] Interface: Wi-Fi
[*] Requested MAC address: 00:11:22:33:44:55
[*] MAC change request validated.
[!] Actual adapter modification is disabled in this demo.

=====
```

Figure 2 — MAC Address Changer

Figure 3 — ARP Spoofing Demo

The module accepts the target IP, gateway IP, and interface, then performs input validation without transmitting spoofing packets.

```
Select an option: 3

[INFO] ARP Spoofing Demo selected.
Enter authorized target IP: 192.168.1.10
Enter gateway IP: 192.168.1.1
Enter network interface name: Wi-Fi

[*] ARP Spoofing Demo
[*] Target IP : 192.168.1.10
[*] Gateway IP: 192.168.1.1
[*] Interface : Wi-Fi
[*] Input validation successful.
[!] Live ARP spoofing is disabled in this demo.
[+] Safe demonstration completed.
```

Figure 3 — ARP Spoofing Demo

Figure 4 — Packet Sniffer Demo

The module accepts the interface and packet limit and demonstrates the configuration without capturing live network traffic.

```
Select an option: 4

[INFO] Packet Sniffer selected.
Enter network interface name: Wi-Fi
Enter packet count for the demo: 4

[*] Packet Sniffer Demo
[*] Interface: Wi-Fi
[*] Packet limit: 4
[*] Input validation successful.
[!] Live packet capture is disabled in this demo.
[+] Safe demonstration completed.
```

Figure 4 — Packet Sniffer Demo

Figure 5 — Network Queue Demo

The module validates the interface and packet limit without performing live packet blocking or jamming.

```
-----
Select an option: 5

[INFO] Network Queue Demo selected.
Enter network interface name: Wi-Fi
Enter packet limit for the demo: 5

[*] Network Queue Demo
[*] Interface: Wi-Fi
[*] Packet limit: 5
[*] Input validation successful.
[!] Live packet blocking/jamming is disabled in this demo.
[+] Safe demonstration completed.
```

Figure 5 — Network Queue Demo

Figure 6 — DNS Spoofing Demo

The module validates the target IP, spoof IP, and interface without intercepting or modifying live DNS traffic.

```
-----
Select an option: 6

[INFO] DNS Spoofing Demo selected.
Enter authorized target IP: 192.168.1.10
Enter demonstration spoof IP: 192.168.1.100
Enter network interface name: Wi-Fi

[*] DNS Spoofing Demo
[*] Target IP : 192.168.1.10
[*] Spoof IP : 192.168.1.100
[*] Interface : Wi-Fi
[*] Input validation successful.
[!] Live DNS interception/modification is disabled in this demo.
[+] Safe demonstration completed.

-----
```

Figure 6 — DNS Spoofing Demo

Figure 7 — ARP Spoof Detector

The defensive detector identifies conflicting MAC addresses associated with the same IP address using controlled test observations.

```
=====
Select an option: 7

[INFO] ARP Spoof Detector selected.

[!] Possible ARP spoofing indicators:
-----
IP Address : 192.168.1.1
Previous MAC: AA:BB:CC:DD:EE:FF
New MAC      : 11:AA:22:BB:33:CC
-----

[!] 1 anomaly/anomalies detected.
```

Figure 7 — ARP Spoof Detector

Network Scanner Evidence

The Network Scanner was tested separately using the authorized local network range configured during testing.

```
Select an option: 2

[INFO] Network Scanner selected.
Enter the authorized target IP or network range: 10.73.59.0/24
[INFO] Scanning target: 10.73.59.0/24
[INFO] Sending ARP requests...
IP Address      MAC Address
-----
10.73.59.129    d0:39:57:30:82:6f
10.73.59.108    32:ef:63:d5:f5:cb

[+] Scan completed. 2 device(s) discovered.
```

Figure 8 — Network Scanner Evidence

Challenges and Improvements

Challenges

During development of the Python Security Toolkit, several implementation challenges were encountered.

1. Windows Network Compatibility

The original classroom scripts included Linux-specific networking commands and configurations. Since the project was developed on Windows 11, the network functionality had to be adapted for the Windows environment.

The Network Scanner also required Npcap to provide the packet access required by Scapy.

2. Hard-Coded Configuration

Some of the original scripts contained fixed IP addresses, interfaces, and other environment-specific values. These were replaced with user-provided inputs wherever applicable.

3. Safe Demonstration of Disruptive Functions

Some of the original cybersecurity concepts can affect network traffic or communication. For the integrated toolkit, these functions were converted into controlled demonstrations rather than performing live disruptive operations.

4. Input Validation and Error Handling

The individual scripts originally required additional validation and consistent error handling. Validation functions and exception handling were added to make the toolkit more reliable.

Improvements

The project was improved by:

- Organizing the scripts into separate modules.
- Creating a central command-line interface.
- Adding reusable utility functions.
- Adding input validation.
- Adding consistent status and error messages.
- Adding automated testing with Pytest.
- Removing unnecessary hard-coded environment values.
- Adding documentation and screenshots.
- Separating potentially disruptive functionality from safe demonstrations.
- Adding a defensive ARP spoof detection module.

Limitations

The current version of the Python Security Toolkit has the following limitations:

1. Several potentially disruptive security functions are implemented as safe demonstrations and don't perform live attack operations.
2. The MAC Address Changer validates the requested MAC address but does not modify the Windows network adapter.
3. The Packet Sniffer demonstrates configuration and validation without performing live packet capture.
4. The Network Queue Demo does not perform live packet blocking, dropping, or modification.
5. The DNS Spoofing Demo does not intercept or modify live DNS traffic.
6. The toolkit is primarily designed for educational and controlled laboratory use.
7. Network-related functionality can depend on the operating system, network interface configuration, privileges, and required dependencies such as Npcap.
8. The ARP Spoof Detector currently analyses supplied observations and demonstrates anomaly detection rather than continuously monitoring a production network.

These limitations are intentional in the current educational implementation and help keep potentially disruptive functionality within controlled testing boundaries.

Conclusion

The Python Security Toolkit successfully integrates multiple cybersecurity scripts into a single organized Python application with a common command-line interface. It demonstrates modular programming, input validation, exception handling, reusable utility functions, automated testing, and structured documentation. The integration of seven security-related modules provides a unified platform for demonstrating different cybersecurity concepts. It also addresses practical development challenges such as Windows compatibility, dependency management, removal of unnecessary hard-coded values, and safe handling of potentially disruptive security functionality. Testing confirmed that the toolkit menu and individual modules operate as intended in the current implementation. The automated validation tests also passed successfully.

Overall, the project provides a structured educational framework for understanding cybersecurity tools while emphasizing authorized testing, controlled laboratory environments, responsible use, and clear technical documentation.

Responsible Use

This toolkit is developed for cybersecurity education, authorized security testing, and controlled laboratory environments.

The modules included in the project can represent techniques that may affect network communication or system behaviour when implemented as live operations. Therefore, the toolkit should only be used on systems, networks, virtual machines, or laboratory environments for which explicit permission has been provided.

The following practices should be followed:

- Use the toolkit only for authorized cybersecurity testing.
- Do not use it against public networks, production systems, or devices belonging to other users without permission.
- Use an isolated laboratory environment whenever possible.
- Do not collect credentials, private communications, or personal data.
- Restore any configuration changes made during authorized testing.
- Keep potentially disruptive security experiments separated from normal or production networks.
- Document the testing environment, dependencies, and limitations.
- Use the toolkit for learning, security assessment, and defensive understanding.

References

1. [Project Guidelines](#) — Python Security Toolkit project requirements and evaluation criteria.
2. [Bootcamp Code Reference](#) — Classroom cybersecurity scripts used as the basis for the integrated modules.
3. [Python Documentation](#) — Python programming language and standard library reference.
4. [Scapy Documentation](#) — Documentation for Python-based packet manipulation and network security functionality.
5. [Pytest Documentation](#) — Documentation for Python testing and automated test execution.
6. [Npcap Documentation](#) — Documentation for Windows packet capture and network access.